RAJALAKSHMI ENGINEERING COLLEGE

Thiviyadaran V 251001292-2025-IT

T2

Started on	Friday, 25 September 2026, 1:55 PM
State	Finished
Completed on	Friday, 25 September 2026, 1:57 PM
Time taken	1 min 30 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

long long max(long long a, long long b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    long long **grid = (long long **)malloc(n * sizeof(long long *));
    long long **dp = (long long **)malloc(n * sizeof(long long *));
    for (int i = 0; i < n; i++) {
        grid[i] = (long long *)malloc(n * sizeof(long long));
        dp[i] = (long long *)malloc(n * sizeof(long long));
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%lld", &grid[i][j]);
        }
    }

    dp[0][0] = grid[0][0];

    for (int i = 1; i < n; i++) {
```

```
    dp[0][j] = dp[0][j - 1] + grid[0][j];
}

for (int i = 1; i < n; i++) {
    dp[i][0] = dp[i - 1][0] + grid[i][0];
}

for (int i = 1; i < n; i++) {
    for (int j = 1; j < n; j++) {
        dp[i][j] = grid[i][j] + max(dp[i - 1][j], dp[i][j - 1]);
    }
}

printf("%lld\n", dp[n - 1][n - 1]);

for (int i = 0; i < n; i++) {
    free(grid[i]);
    free(dp[i]);
}
free(grid);
free(dp);

return 0;
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course